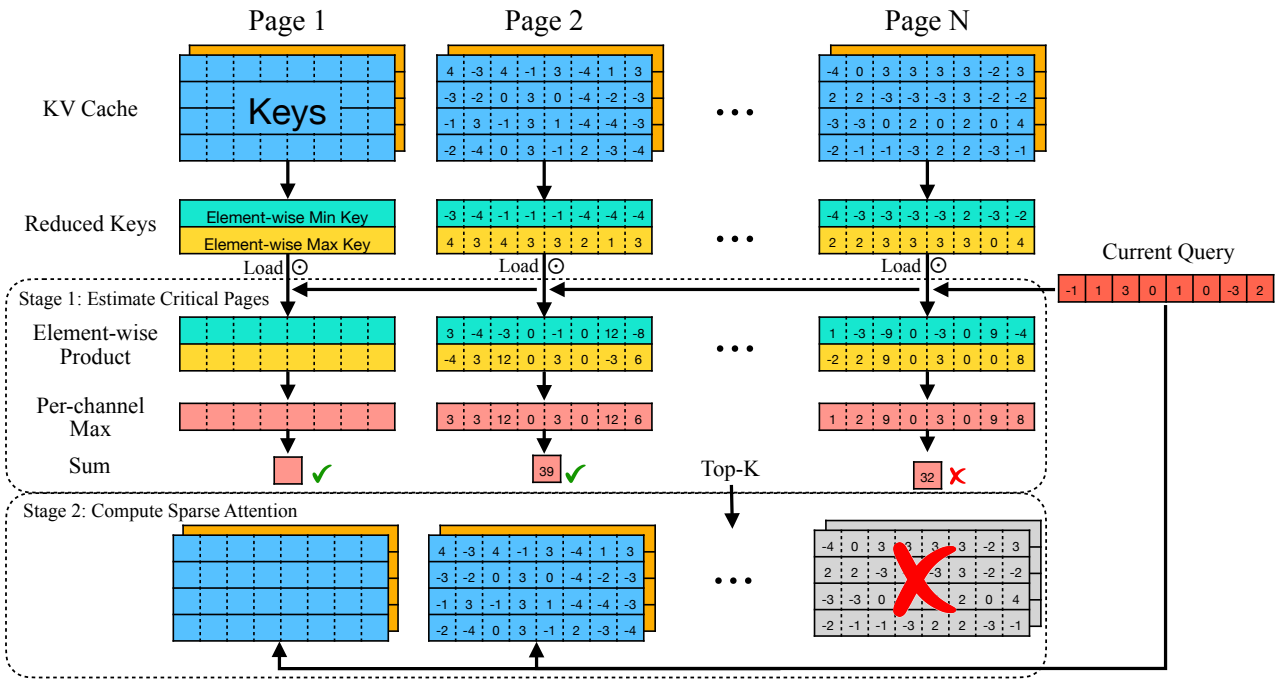

Figure 5. Quest performs self-attention in two stages. In stage 1, Quest estimates the criticality of pages by performing element-wise product between the current Query vector and both Min Key and Max Key vectors in each KV cache page. Quest gets the sum of the per-channel maximal value for each page as the page criticality estimation. In stage 2, only Top-K KV cache pages are loaded to perform sparse self-attention with the current Query.

rates since critical tokens are pruned in previous iterations. As shown in Fig. 4, Quest maintains recall rate close to full attention, as it estimated critical tokens based on current query. Therefore, pre-determining the criticality is challenging, which motivates query-aware sparsity by considering Q vectors for criticality estimation.

3.4. Dynamically Estimating Token Criticality

To efficiently and accurately estimate the criticality of KV cache tokens, we propose Quest, an efficient and accurate algorithm that exploits query-aware context sparsity, which approximately selects the most potentially critical KV cache pages for the current query. We show the workflow of Quest in Fig. 5. To manage the overhead, Quest adopts PageAttention (Kwon et al., 2023) and selects the KV cache pages at the granularity of pages.

To estimate the criticality of the pages, Quest performs an approximate calculation of attention weights before the original attention operation, as shown in Algorithm 1.

Our insight is that in order not to miss critical tokens, we should select pages containing the token with the highest attention weights. However, for an efficient selection of pages, we should calculate an approximate attention score following this insight. We found that the upper bound attention weights within a page can be used to approximate the highest attention in the page. The upper bound of the attention weights can be calculated by the channel-wise minimal values (m_i) and maximal values (M_i) of Key vectors. Given a Q vector, Quest calculates the maximum possible value of the channel i by taking $U_i = \max(Q_i m_i, Q_i M_i)$. Note that U_i is always greater than any product of Q_i with the Key value K_i for all tokens in this page regardless of the sign of Q_i. Therefore, when we add up U_i, we get the upper bound of attention weights across all Key vectors on this page.

After deriving the upper bound attention weights, we choose the top K pages as critical, where K is an arbitrarily defined hyper-parameter. To demonstrate the feasibility of Quest, we perform actual self-attention and gather Top-K per-page attention scores. As shown in Fig. 3, our query-aware sparsity mostly aligns with the oracle sparsity. Quest performs normal self-attention only on selected pages, which greatly reduces memory movement. We define the number of tokens in selected pages as the "Token Budget".

Due to the low sparsity ratio for the first two layers (as shown in Fig. 3), we only apply Quest and all baselines on later layers to better preserve model accuracy. Note that whether to skip the first two layers or not is orthogonal to the KV cache selection algorithm.

Algorithm 1 Token Criticality Estimation

When inserting new token to KV cache:
Input: Key vector K, Dimension of hidden states dim, Current maximal vector M_i, Current minimal vector m_i

for $i = 1$ **to** dim **do**
  M_i = $\max(M_i, k_i)$
  m_i = $\min(m_i, k_i)$
end for

When perform self-attention:
Input: Query vector Q, Dimension of hidden states dim, Current maximal vector M_i, Current minimal vector m_i

Initialize $score = 0$.
for $i = 1$ **to** dim **do**
  $score$ += $MAX(q_i * max, q_i * min)$
end for

3.5. Quest Reduces the Memory Movement of Self-Attention

Instead of loading the whole KV cache, Quest only needs to load a fraction of the data, which leverages query-aware sparsity. Assume that every K or V vector is M bytes, the KV cache contains L tokens, and each page contains S KV pairs (Page size). During criticality estimation, Quest will load maximal and minimal vectors of each page, which is approximately $2M * L/S$ bytes. Additionally, Quest performs normal self-attention for top K pages, which is $2M * K * S$ bytes. The whole KV cache is $2M * L$ bytes, which indicates Quest loads $1/S + K * S/L$ of the total KV cache[‡], which is equivalent to

$$\frac{1}{\text{Page Size}} + \frac{K}{\text{Page Num}}$$

Assuming that we use 16 KV pairs per page, context length is 64K, and we choose the top 4K pages, Quest will reduce the memory load by $8\times$. Note that this memory load reduction is universal across all models and is compatible with existing quantization mechanisms (Zhao et al., 2024).

4. Experiments

4.1. Setting

We evaluate Quest on the language modeling dataset PG19 (Rae et al., 2019), passkey retrieval task (Peng et al., 2023), and six datasets in LongBench (Bai et al., 2023): NarrativeQA (Kočiský et al., 2018), HotpotQA (Yang et al., 2018), Qasper (Dasigi et al., 2021), TrivialQA (Joshi et al., 2017), GovReport (Huang et al., 2021), MultifieldQA (Bai et al., 2023). We choose two widely used long-context models for our evaluation: LongChat-v1.5-7b-32k (Li et al., 2023) and Yarn-Llama-2-7b-128k (Peng et al., 2023). We compare our method against the KV cache eviction algorithm H2O (Zhang et al., 2023b), TOVA (Oren et al., 2024), and StreamingLLM (Xiao et al., 2023). Note that we **do not** apply any Quest and other baseline algorithms to the first two layers of the model, as our analysis in Sec 3.4 indicates a low sparsity ratio for these layers.

[‡]The top-K operator incurs negligible memory loading and execution time (5-10 us). Therefore, we do not include it in efficiency analysis.

Method / Budget	32	64	128	256	512
H2O	0%	1%	1%	1%	3%
TOVA	0%	1%	1%	3%	8%
StreamingLLM	1%	1%	1%	3%	5%
Quest (ours)	**65%**	**99%**	**99%**	**99%**	**100%**

Method / Budget	256	512	1024	2048	4096
H2O	2%	2%	2%	2%	4%
TOVA	2%	2%	2%	2%	10%
StreamingLLM	1%	1%	1%	2%	4%
Quest (ours)	**88%**	**92%**	**96%**	**100%**	**100%**

Table 1. (i) Results of 10k length passkey retrieval test on LongChat-7b-v1.5-32k. (ii) Results of 100k length passkey retrieval test on Yarn-Llama-2-7b-128k. Quest can achieve nearly perfect accuracy with 64 and 1024 tokens KV cache budget, which is about 1% of the total sequence length, demonstrating that Quest can effectively preserve the model's ability to handle long-dependency tasks. However, KV cache eviction algorithms such as H2O, TOVA, and StreamingLLM incorrectly discard the KV cache of the answer before receiving the question, thus failing to achieve ideal accuracy.

4.2. Accuracy Evaluation

4.2.1. Language Modeling on PG19

We first evaluate the language modeling perplexity on the PG19 test set, which is a dataset comprising 100 books with an average length of 70k tokens. We use the LongChat-7b-v1.5-32k model to test 32k tokens on PG19. We feed the model with various numbers of tokens and evaluate the perplexity of generated tokens. We evaluate H2O, TOVA, and Quest with a token budget of 4096, which is approximately 1/8 of the total token length. As indicated by the perplexity results in Fig. 6, Quest's accuracy closely matches the oracle baseline with a full KV cache.

4.2.2. Results on long text passkey retrieval task

Since language modeling evaluation only involves local dependencies, models can achieve great performance by fo-

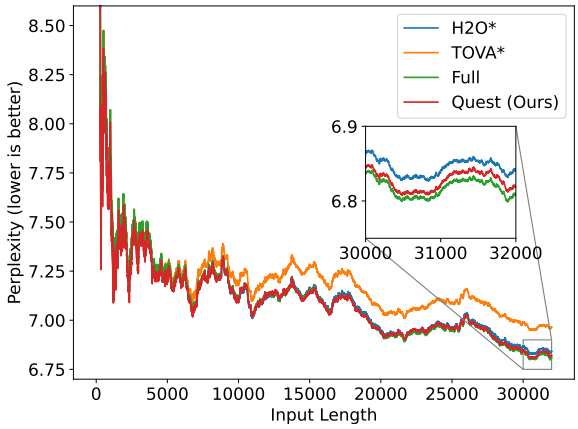

Figure 6. Language modeling evaluation of Quest on PG19 dataset. We prompt the model with 0 to 32000 tokens from the PG19 test set and measure the perplexity of output tokens. H2O* and TOVA* indicate that for the first two layers of models, we do not apply these two algorithms to prune the KV Cache, as analyzed in Sec 3.4, which better preserves the model performance. Quest also uses a full cache in the first two layers of the model. Quest can closely match the performance of the full cache model.

cusing on recent tokens. However, the ability to handle long-distance dependencies is crucial for long text reasoning. For KV cache eviction algorithms like H2O and TOVA, parts of KV caches that are important for distant future tokens may be discarded, thereby preventing the model from obtaining the correct answer. To show that Quest helps maintain the ability of models to handle longer dependency tasks, we evaluate it on the passkey retrieval task from Yarn (Peng et al., 2023). This task measures a model's ability to retrieve a simple passkey from a large amount of meaningless text. We put the answer in different depth ratios of the text and evaluate if the model can retrieve the correct answer with different KV cache token budgets. We evaluate LongChat-7b-v1.5-32k on 10k tokens test and Yarn-Llama-2-7b-128k on 100k tokens test.

Since H2O (Zhang et al., 2023b) needs to calculate historical attention scores for KV cache pruning, it needs to compute the complete $O(n^2)$ attention map and thus is unable to use Flash-Attention (Dao et al., 2022) for long-context inference. Therefore, to enable H2O on long-context evaluation, we use Flash-Attention in the context stage for the 100k sequence length passkey retrieval test and start collecting historical attention scores for H2O in the decoding stage. For TOVA (Oren et al., 2024) and StreamingLLM (Xiao et al., 2023), we evaluated them on the 10k and 100k sequence lengths.

For the passkey retrieval test, we directly prefill the input text containing the passkey and texts to the model. However, to evaluate the impact of different methods on the model's ability to handle long-dependency tasks in practical scenarios, we simulate decoding by feeding the task's question and instruction to the model token by token. In this case, H2O and TOVA might mistakenly discard tokens critical for future tokens, such as the passkey that will be queried later. Similarly, StreamingLLM can only focus on the most recent text window, and if the passkey appears outside this window, it cannot provide the correct answer. Therefore, H2O, TOVA, and StreamingLLM cannot achieve ideal accuracy on the 10k and 100k length passkey retrieve test. However, Quest does not discard KV cache but instead uses a query-aware approach to identify critical tokens. As shown in Tab. 1, Quest can achieve perfect accuracy with a minimal budget both on 10k and 100k sequence length tests.

4.2.3. Results on LongBench

To validate that Quest can outperform baselines on general long-context datasets, we evaluate our method and baselines on six datasets in LongBench. We evaluate on LongChat-7b-v1.5-32k across a wide range of long-context datasets, including single-document QA: NarrativeQA, Qasper, MultiFieldQA; multi-document QA: HotpotQA; summarization: GovReport; few-shot learning: TriviaQA. We evaluate H2O, TOVA, StreamingLLM, and Quest with different KV cache budgets. For all datasets, we split the input into material and question/instruction. For the material part, we use Flash-Attention (Dao et al., 2022) with the full KV cache to perform inference. For the question part, we simulate decoding by feeding them to the model token by token. Similar to the passkey retrieval test, to enable H2O to use Flash-Attention, we could not collect H2O's historical attention scores during the context stage, thus starting from the decoding stage.

As shown in the Fig. 7, Quest consistently outperforms all baselines across six long-context datasets with various KV cache budgets. Quest with a budget of 1K tokens can achieve comparable performance as the model with full KV cache, while other baselines still exhibit a notable gap from full cache performance even with a larger budget. After considering the full cache used in the first two layers, Quest can achieve lossless performance on Qasper, HotpotQA, GovReport, TriviaQA, NarrativeQA, and MultifieldQA with KV cache sparsity of 1/6, 1/6, 1/5, 1/10, 1/5, and 1/6, respectively. This demonstrates that Quest is capable of maintaining the model's capabilities across different types of long-context tasks, as it does not lead to the generation of incorrect answers due to improper discarding of KV cache.

4.3. Efficiency evaluation

To demonstrate the feasibility of Quest, we implement the entire framework with dedicated CUDA kernels based on FlashInfer (Ye et al., 2024), a kernel library for LLM inference. We first evaluate Quest's kernel-level efficiency